

星汉：面向 AI 智能体的内核态域名感知透明路由系统

作者

ASTRA: An In-Kernel Domain-Aware Transparent Routing System for AI Agents

Abstract With the rapid advancement of large language model (LLM)-driven AI agents, tool use has become the dominant paradigm through which agents dynamically interact with external services during task execution. In multi-tenant agent platforms, each agent sandbox often needs to access a wide range of heterogeneous external services concurrently, including LLM inference APIs, web scraping APIs, code space APIs, and financial data APIs. These APIs often require distinct network egress paths. Existing solutions each have significant limitations: tunnel-based approaches require establishing a separate channel per service, leading to maintenance costs that grow sharply with the number of services; IP-based policy routing struggles with the dynamically changing endpoints of API providers; and user-space proxies such as HAProxy and Envoy incur context-switching overhead and require intrusive configuration. To address these challenges, this paper proposes ASTRA, a transparent, high-performance name-based routing system. ASTRA leverages the existing TLS Server Name Indication (SNI) extension as a service identifier, enabling transparent routing compatible with the existing ecosystem without requiring modifications to applications or infrastructure. Meanwhile, it implements domain-name-based routing decisions entirely within the Linux kernel, eliminating the context-switching overhead of user-space proxies to achieve high-performance forwarding. ASTRA incorporates three key techniques: (1) a cross-layer name-based routing scheme using the combination of TLS server name and TCP port as the service identifier; (2) a 4-way TCP-TLS joint handshake proxy model that transparently intercepts connections and extracts service names in kernel space; and (3) a deferred in-kernel name resolution mechanism based on a custom radix tree for efficient name-to-address translation. Experimental results show that, under typical API payload sizes, ASTRA outperforms HAProxy and Envoy in QPS by 27.16% and 50.74%, and in P50 latency by 17.68% and 43.60%, respectively, while maintaining near-constant lookup latency as name rules scale from 1,000 to 50,000, showing scalability as the number of name rules grows.

Key words AI agents; name-based routing; multi-source service access; Linux kernel; deferred name resolution

摘要 随着大语言模型驱动的 AI 智能体技术的快速发展,智能体通过工具调用机制在任务执行过程中动态访问外部服务已成为主流范式。在多租户智能体平台中,每个智能体沙箱往往需要同时访问大量异构外部服务,如大模型推理 API、网页抓取 API、代码仓库 API 及金融数据 API 等。这些 API 往往需要经由不同的网络出口才能可达。传统方案中,隧道方案需要为每个服务分别建立通道,维护成本随服务数量急剧增长;基于 IP 地址的策略路由方案则因 API 接入点 IP 地址的动态变化而难以维护;用户态代理(如 HAProxy、Envoy)

收稿日期: ; 修回日期:

基金项目:

通信作者:

存在上下文切换开销且需要侵入式配置。针对上述问题,本文提出星汉系统,一种透明、高性能的按名路由系统。星汉系统利用 TLS 握手中已有的 SNI (Server Name Indication) 扩展字段作为服务标识,无需修改应用或基础设施即可实现与现有生态兼容的透明路由;同时将基于服务域名的路由决策完全置于 Linux 内核中执行,消除用户态代理的上下文切换开销,实现高性能转发。星汉系统包含三项关键技术:1) 跨层按名路由方案,以 TLS 服务名和 TCP 端口的组合作为服务标识符;2) 四路 TCP-TLS 联合握手代理模型,在内核态透明地拦截连接并提取服务名;3) 延迟内核态名字解析机制,基于定制的基数树实现高效的名字-地址转换。实验结果表明,在典型 API 负载大小下,星汉系统的 QPS 超过 HAProxy 27.16%、超过 Envoy 50.74%, P50 延迟低于 HAProxy 17.68%、低于 Envoy 43.60%,且在名字规则从 1,000 增长到 50,000 时保持几乎恒定的查找延迟,随名字规则数量的增长保持良好的可扩展性。

关键词 AI 智能体;按名路由;多源服务接入;Linux 内核;延迟名字解析

中图法分类号 TP311.13; TP309

随着大语言模型 (Large Language Model, LLM) 技术的快速演进, AI 智能体 (AI Agent) 正在从单一的对话系统发展为具备自主决策和工具调用能力的复合系统^[1,2]。以 OpenClaw^[3]、Claude Code^[4] 等为代表的智能体框架通过工具调用 (tool use)^[5] 机制,使智能体能够在执行任务的过程中动态访问外部服务。研究表明,单个智能体任务可能涉及多达 16,000 种 API 中的 7 至 15 个不同调用^[6]。例如调用 LLM 推理 API 获取自然语言理解能力,调用网页抓取 API (如 Firecrawl^[7]) 获取实时信息,调用代码托管 API (如 GitHub^[8]) 完成代码操作,调用金融数据 API 获取市场行情等。模型上下文协议 (Model Context Protocol, MCP)^[9] 的提出进一步标准化了智能体与外部服务的交互接口,使智能体可访问的服务种类和数量持续增长。

这种多源服务接入模式给网络基础设施带来了新的挑战。与传统 Web 应用访问单一后端不同, AI 智能体沙箱 (sandbox) 往往需要同时访问分布在不同网络域的多个异构 API 端点,且这些端点的网络可达性要求各不相同。具体而言:1) 云厂商的 LLM 推理服务 (如 AWS Bedrock^[10]、Azure OpenAI Service^[11] 或 Google Vertex AI^[12]) 通常需要通过其专有虚拟网络 (Virtual Private Cloud, VPC) 访问,要求流量经由特定的 VPC 端点或专线出口;2) 网页抓取类 API (如 Firecrawl) 对出口 IP 的质量和地理位置有严格要求,需要通过特定 IP 地址池出口;3) 金融数据 API (如交易所数据) 往往部署在专用网络中,需要通过专线或 VPN 接入;4) 代码托管 API 和协作类 MCP Server (如 GitHub Enterprise、GitLab 私有实例等) 可能部署在企业内网中,需要通过企业网关访问。在多租户智能体平台中 (如 Manus^[13]), 数百乃至数千个智能体沙箱并发运行,每个沙箱都可能需要同时访问上述部分或者所有类型的服务,使得差异化路由的需求更为突出。

针对多源服务场景,现有的网络接入方案存在以

下局限性:

1) 隧道方案维护成本高。最直接的方法是为每个需要差异化路由的服务建立独立的 VPN 隧道 (如 WireGuard^[14]、IPsec^[15] 等)。这意味着 AI 智能体沙箱每接入一个新的 API 服务提供商,就需要配置一条新的隧道,并维护对应的密钥、路由规则和连接状态。当服务数量增长到数十甚至上百时 (这在 AI 智能体的工具调用场景中并不罕见), 隧道的配置和运维复杂度将急剧增长。此外,在多租户平台中,为了实施网络隔离,每个沙箱都需要维护独立的隧道集合。若平台同时运行 N 个沙箱、接入 M 个服务提供商,则系统需维护的隧道总数为 $O(N \times M)$,这在运维上是难以接受的。

2) 策略路由难以应对动态端点。另一种方案是在网关处配置基于目的 IP 地址的策略路由,将不同 IP 地址段的流量导向不同的网络出口。然而,现代云服务和 API 提供商的接入点 IP 地址并非固定不变:云厂商会频繁调整其服务的 IP 地址段^[16], CDN 服务的解析结果因地理位置和负载状态而异^[17],部分 API 提供商甚至不公开其 IP 地址范围。这意味着策略路由规则需要实时更新以跟踪这些变化。一旦规则滞后,流量将被发送到错误的出口,导致服务不可达。

3) 用户态代理存在性能瓶颈与侵入性配置问题。在实践中,基于域名的路由通常通过应用层代理实现,如 HAProxy^[18]、Nginx^[19] 和 Envoy^[20]。这些代理运行在用户空间,能够解析 TLS SNI 或 HTTP Host 字段来进行路由决策。然而,用户态代理需要终止并重建 TCP 连接,数据需要在用户空间和内核空间之间进行两次拷贝和上下文切换。在多租户智能体平台的高并发场景下 (数千个沙箱同时发起 API 调用), 这一开销显著降低了吞吐量并增加了端到端延迟。此外,部分代理方案还需要智能体应用显式配置代理地址,这对智能体的透明运行构成了侵入。

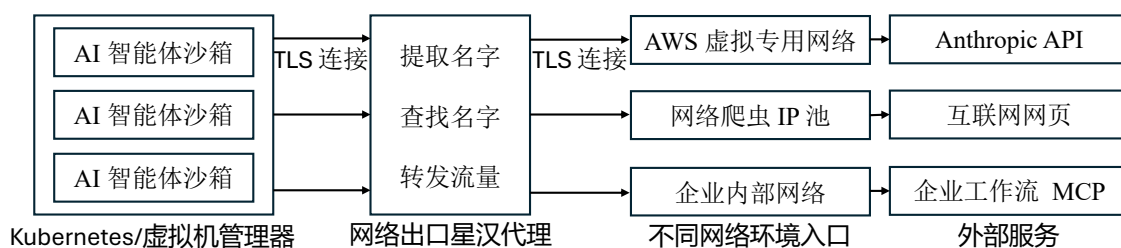


Figure 1 A typical multi-source API access scenario supported by ASTRA: two ASTRA proxies transparently rewrite destination addresses based on TLS SNI, routing requests from the agent sandbox to the AWS Bedrock server via a dedicated private link, with responses returned along the reverse path

图 1 星汉系统支持的典型 AI 智能体多源 API 接入场景: 两级星汉代理根据 TLS SNI 透明改写目的地址, 将智能体沙箱的请求经专线路由至 AWS Bedrock 服务器, 响应沿反向路径返回

上述分析表明, 现有工作面临两方面的根本矛盾。其一是路由精确性问题: API 服务往往以域名作为其稳定标识, 但传统路由系统只能基于 IP 地址进行转发决策。在现有的域名系统 (Domain Name System, DNS) 中, 域名到 IP 地址的映射是动态的、多对多的, 这一语义在路由决策时已经丢失。隧道方案和策略路由方案均受制于此——前者依赖 IP 地址段划分隧道, 后者直接以 IP 前缀作为分流依据, 两者都无法稳定地表达“按服务名差异化路由”的语义。其二是路由透明性问题: 虽然用户态代理能够解析 TLS SNI 或 HTTP Host 字段, 但其要求应用显式配置代理地址。在智能体场景中, 工具调用链涉及智能体框架、MCP 客户端、HTTP 库等多层软件栈, 任何一层引入代理配置都可能导致兼容性问题; 同时, 智能体可调用的服务种类随 API 服务和 MCP 生态的扩展而持续增长, 逐一适配每个工具库的代理设置在工程上不可持续。因此, 理想的按名路由系统应同时满足两个条件: **能够直接基于域名进行转发决策, 且在网络层透明地完成路由**, 使上层软件栈无需感知路由策略的存在。

针对上述挑战, 本文提出了星汉系统 (Agent Service Transparent Routing Architecture, ASTRA), 一种在 Linux 内核中实现的、基于域名的透明路由系统, 使 AI 智能体沙箱无需修改应用代码即可根据目标服务域名将流量差异化路由到不同网络出口, 如图 1 所示。星汉系统的主要贡献包括以下 3 个方面:

1) 跨层按名路由方案: 利用 TLS SNI 将服务标识与网络位置解耦, 使智能体沙箱能够基于服务域名而非 IP 地址进行差异化路由, 同时无需修改现有网络架构;

2) 四路 TCP-TLS 联合握手代理模型: 在 Linux 内核中实现透明的连接拦截和服务名提取, 避免了用户态代理的上下文切换开销;

3) 延迟内核态名字解析机制: 基于定制的基数树

实现高效、可扩展的名字到地址转换, 使名字解析与路由决策在内核态一体化完成。

1 背景与动机

1.1 AI 智能体的多源服务接入需求

AI 智能体是一种以大语言模型为核心, 通过感知环境、自主推理和调用外部工具来完成复杂任务的智能系统^[1]。与传统的单轮问答式 AI 应用不同, 智能体在执行任务过程中通常需要多步骤地、动态地与物理世界交互, 并访问各种外部服务。以一个典型的信息研究智能体工作流为例: 智能体首先调用大模型推理 API (如 OpenAI GPT、AWS Bedrock 上的 Claude) 进行任务分解和推理; 随后调用网页抓取 API (如 Firecrawl) 获取目标网页内容; 接着调用代码托管 API (如 GitHub API) 检索相关代码仓库; 在分析过程中可能还需调用金融数据 API (如 Wind、Bloomberg) 获取实时行情数据; 最后再次调用大模型 API 对收集到的信息进行汇总和生成报告。整个流程中, 智能体可能访问 5-10 个甚至更多不同的外部服务 API。这一模式已在多个系统中得到验证: SWE-agent^[21] 在处理 GitHub Issue 时需同时访问代码仓库、shell 终端和测试框架; MetaGPT^[22] 的多智能体协作流程中, 不同角色的智能体分别调用 Web 搜索、代码执行和文件系统等服务。

近年来, 智能体框架 (如 OpenClaw^[3]、LangChain^[23]、MetaGPT^[22]、OpenHands^[24]) 和模型上下文协议 (MCP)^[9] 的兴起使智能体可调用的服务种类不断扩大。MCP 协议定义了一套标准化的接口, 使智能体能够以统一的方式访问文件系统、数据库、API 服务等各类外部资源。相较于此前每个外部服务都需要编写定制化适配代码的方式, MCP 显著降低了服务接入的开发门槛, 这使得智能体可调用的服务种类不断扩大。截至 2026 年 3 月, 社区 MCP 服

务目录已收录近 20,000 个服务端点^[25]。

在多租户 AI 智能体平台中，多源服务接入带来的路由挑战将进一步放大。平台通常为每个用户或每个任务分配一个独立的沙箱（sandbox）环境以实现隔离。例如 OpenHands^[24] 为每个任务分配独立的 Docker 容器在高峰期，平台上可能同时运行数百甚至数千个沙箱，每个沙箱都独立发起对多个外部 API 的请求。这种大规模并发的多源服务接入模式，对底层网络路由架构提出了严峻的挑战。

1.2 差异化网络出口的必要性

上述 API 服务并非都能通过统一的互联网出口直接访问，它们各自有着不同的网络可达性约束，这是多源服务接入问题的核心所在。

1) 云厂商专有的网络约束。主流云厂商的 AI 推理服务（如 AWS Bedrock、Azure OpenAI Service、Google Vertex AI）通常部署在其私有网络内部，推荐甚至强制要求通过其云服务的 VPC 端点（VPC Endpoint）或私有链路（PrivateLink）^[26]访问。直接通过公共互联网访问这些服务可能面临更高的延迟、安全合规限制，或者根本不可达。因此，访问这些服务的流量需要经由与对应云厂商建立了网络互联的专线连接或出口转发。

2) 出口 IP 质量约束。网页抓取类 API 和部分数据服务对请求来源 IP 的质量或地理位置有严格要求。许多网站和 API 会对来自数据中心 IP 段的请求实施限速或封禁^[27]。为确保抓取服务的可用性，这类 API 往往需要通过住宅 IP 代理（residential proxy）或经过特定 IP 声誉处理的出口访问。

3) 专线/合规约束。金融数据 API、医疗健康 API 等受监管行业的服务通常部署在私有网络中，要求通过专线（如 MPLS^[28]、SD-WAN^[29] 专线）或经过合规审计的 VPN 通道接入。部分服务可能对源 IP 地址进行白名单过滤，仅允许来自预授权 IP 段的访问。

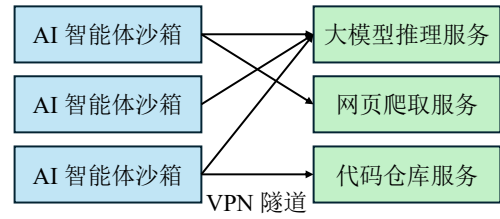
4) 企业内网约束。企业自建的私有服务（如 GitLab 私有实例、Jira、内部知识库 API 或是用于内部协作的 MCP 服务器）部署在企业内网中，需要通过企业 VPN 或零信任网关（如 Cloudflare Access^[30]）才能访问。

以上约束意味着，从一个智能体沙箱发出的流量需要根据目标服务的不同，被路由到不同的网络出口。因此，底层网络路由架构需要解决上述按服务名差异化路由的问题。

1.3 现有工作的局限性

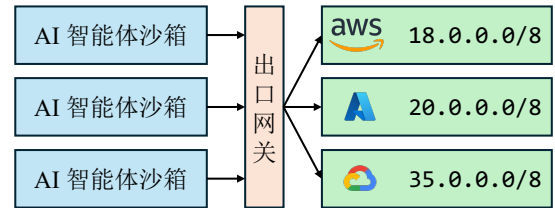
为实现上述差异化路由，学术界和工业界存在如下几类方案。

1) 隧道方案（Tunnel-based）。隧道方案（如



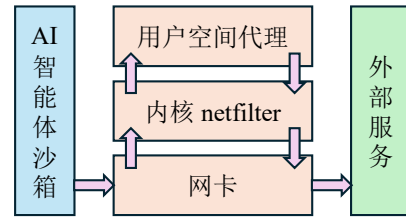
(a) Tunnel-based solution: each service requires a dedicated tunnel, with maintenance costs growing linearly with the number of services

(a) 隧道方案：每个服务需独立建立隧道，维护成本随服务数量线性增长



(b) Policy-based routing solution: dynamic changes in API endpoint IP ranges cause routing rules to become stale, leading to traffic misrouting

(b) 策略路由方案：API 端点 IP 范围动态变化导致路由规则失效，流量被错误转发



(c) User-space proxy solution: data undergoes two user-kernel space copies and context switches, resulting in a performance bottleneck

(c) 用户态代理方案：数据经历两次用户—内核空间拷贝与上下文切换，产生性能瓶颈

Figure 2 Limitations of existing multi-source service access approaches

图 2 现有多源服务接入方案的局限性

WireGuard^[14]、OpenVPN^[31]）为每个需要差异化出口的服务建立一条独立的加密隧道，并通过策略路由将匹配特定目的 IP 地址段的流量导入对应隧道接口。该方案的主要问题在于可扩展性差：每新增一个服务提供商就需要新建一条隧道并配置对应的路由规则，管理复杂度与服务数量成线性关系。在多租户场景下，若每个沙箱都需要维护独立的隧道集合，总的隧道数量将达到沙箱数量与服务数量的乘积，这在运维上是不可接受的，如图 2（a）所示。此外，隧道方案本质上仍然依赖 IP 地址进行路由，无法解决下文所述的 IP 动态变化问题。隧道方案的维护成本随服务数量线性增长，无法满足智能体多源服务接入的可扩展性

需求。

2) 策略路由方案 (Policy-based Routing)。策略路由通过在网路上配置基于目的 IP 地址前缀的规则,将不同 IP 地址段的流量分流到不同出口。然而,云服务和 API 提供商的 IP 地址池是动态变化的^[16],且部分服务通过 CDN 分发,DNS 解析结果将因时因地而异。策略路由规则无法自动跟踪这些变化,一旦规则与实际 IP 发生偏移,流量将被错误路由,如图 2 (b) 所示。尽管可以编写脚本定期更新规则,但这引入了额外的复杂性和故障点,且存在更新时间窗口内的路由不一致问题。更根本的问题是,IP 地址与服务之间不存在稳定的一一对应关系——同一 IP 可能承载多个服务(虚拟主机),同一服务也可能对应多个 IP (负载均衡/CDN),这使得基于 IP 的差异化路由在语义上就是不精确的。基于 IP 地址的策略路由无法有效追踪 API 端点的动态变化,在语义上也无法精确表达“按服务名路由”的需求。

3) 用户态代理方案 (User-space Proxy)。应用层代理(如 HAProxy^[18]、Envoy^[20]、Nginx^[19])运行在用户空间,能够解析 TLS SNI 或 HTTP Host 字段,实现按名路由。然而,用户态代理的性能受到用户-内核空间数据拷贝和上下文切换的制约。每个代理连接需要经历两次内核-用户空间的数据传输,在高并发场景下这一开销不可忽视,如图 2 (c) 所示。此外,许多代理方案要求客户端应用显式配置代理(如设置 HTTP_PROXY/HTTPS_PROXY 环境变量或 SOCKS 代理),这对于需要透明运行的 AI 智能体应用构成侵入。即使采用透明代理模式(如 iptables REDIRECT + TPROXY),数据路径仍需穿越用户空间。用户态代理的上下文切换和内存拷贝开销在多租户高并发场景下构成显著的性能瓶颈,且侵入式配置破坏了智能体的透明运行。

4) 服务网格方案 (Service Mesh)。以 Istio/Envoy 为代表的服务网格方案^[20],^[32]通过在每个 Pod/容器旁部署 Sidecar 代理来实现流量管理。虽然服务网格能够基于域名进行路由,但其设计初衷是面向微服务间通信的东西向流量,而非智能体到外部 API 的南北向流量。同时,服务网格的 Sidecar 模式同样存在用户态代理的性能开销问题,且引入了 Pilot/Control Plane 等复杂组件,对于仅需要“按名选择网络出口”的场景而言,在每个用户的沙箱旁边同时运行一个 Sidecar 开销过大。服务网格方案对于仅需按名选择网络出口的智能体场景而言过于重量级,其 Sidecar 代理同样受限于用户态的性能开销。

1.4 星汉系统的设计机遇

综合上述分析,现有工作的根本局限在于路由决策层缺乏服务名信息。具体而言,问题可归结为以下三点:

1) IP 地址不是最合适的服务标识。在 AI 智能体的多源服务接入场景中,服务以域名作为其稳定标识,而 IP 地址仅是域名的一个动态映射结果。基于 IP 地址的路由方案无法准确、稳定地表达“按服务名路由”的语义。

2) 路由信息不足。传统路由系统(无论是内核路由表还是 netfilter)仅能访问 L3 (IP 地址)和 L4 (协议端口号)信息。服务名位于更高的协议层 (L6 的 TLS SNI 或 L7 的 HTTP Host),传统路由系统无法直接利用这些信息做出决策。

3) 高兼容性要求。对于 AI 智能体沙箱的场景,解决方案不应要求修改智能体应用代码、主机协议栈或网络基础设施(如路由器、防火墙),以确保对现有生态系统的兼容性。

针对上述三点需求,本文提出星汉系统,并在设计上逐一给出对应的解决方案:

1) 身份-位置分离 (Identity-Locator Separation): 针对需求 1),使用 TLS SNI 中携带的服务名(域名)而非 IP 地址进行路由决策,利用名字空间的语义信息实现精确的差异化路由;

2) 内核态路由决策 (In-Kernel Routing): 针对需求 2),将 TLS SNI 的解析与路由决策完全置于 Linux 内核的 netfilter 框架中执行,使路由系统能够直接获取高层协议中的服务名信息,同时避免用户态代理的上下文切换开销;

3) 透明性 (Transparency): 针对需求 3),星汉系统对智能体应用、网络协议栈和中间设备完全透明,无需修改现有软件栈或引入额外组件。

图 2 展示了星汉系统的基本工作原理。当智能体发起对 bedrock.us-west-1.amazonaws.com 的 HTTPS 请求时,星汉系统在内核中拦截 TLS 握手,提取 SNI 字段获取服务域名,在内核态名字规则表中查找匹配规则(如 *.amazonaws.com → AWS 出口),并将数据包的目的地址改写为对应出口网关的地址,实现按名差异化路由。整个过程对智能体应用完全透明。

2 相关工作

2.1 按名路由的演进

按名路由 (name-based routing) 旨在将路由决策从 IP 地址空间提升到名字空间,以实现更灵活、更精确的流量调度。

早期的学术工作如 IPNL[33] 和 HIP[34], [35] 尝试将服务标识与网络位置解耦。IPNL 引入了一个 L3.5 层头部, 需要修改主机操作系统的网络栈; HIP 则引入了主机身份协议层, 不仅需要修改主机, 还要求路径上的中间设备(防火墙、NAT 等)支持新协议。这些方案的部署门槛极高。全新架构方案如信息中心网络 (Information-Centric Networking, ICN) [36]和命名数据网络 (Named Data Networking, NDN) [37]彻底用内容名替代 IP 地址进行路由, 但与现有 IP 基础设施不兼容, 难以增量部署。星汉系统则利用已广泛应用的 TLS SNI 实现类似的标识-位置解耦, 无需任何基础设施升级。

2.2 基于 IP 的路由方案

隧道方案 (如 WireGuard^[14]) 通过建立加密隧道连接不同网络域, 但在需要同时访问多个异构服务时面临“每服务一隧道”的管理负担。策略路由通过 IP 地址前缀匹配进行分流, 但无法追踪 API 端点 IP 的动态变化。覆盖网络方案 (overlay network, 如 VXLAN[38]) 通过构建虚拟 IP 层来统一地址空间, 但需要在每个节点部署代理并维护全局 IPAM 系统, 部署成本过高。这些 IP 层方案的共同问题在于, 它们试图在 IP 地址层面解决本质上属于服务标识层面的路由问题。

2.3 应用层代理与服务网格

在实践中, 基于名字的路由有时通过用户态应用层代理实现。HAProxy^[18] 和 Nginx^[19] 是广泛部署的 L4/L7 负载均衡器, 能够通过解析 TLS SNI 或 HTTPHost 字段实现按名路由。然而, 它们运行在用户空间, 数据需要在用户-内核空间之间往返拷贝, 导致吞吐量降低和延迟增加。Envoy^[20] 是 Istio 服务网格的默认数据面代理, 设计目标是服务间流量管理。Envoy 支持基于 SNI 的路由[39], 但同样运行在用户态。在服务网格架构中, Envoy 通常以 Sidecar 模式部署在每个 Pod 旁, 所有出入流量都经过 Sidecar 代理。这种架构虽然功能丰富 (支持可观测性、故障注入、流量镜像等), 但对于“仅需根据域名选择网络出口”的智能体接入场景而言过于复杂, 且 Sidecar 的资源开销在大规模沙箱的部署中不可忽视。

近期的一些优化工作[40], [41] 聚焦于特定场景的代理性能提升, 如利用可编程交换机加速 L7 负载均衡、利用 XDP 加速 SYN Cookie 处理等, 但尚未解决通用的按名内核态路由问题。

星汉系统将代理逻辑移入内核, 结合了名字路由的灵活性与 L4 转发的高性能, 同时对应用程序完全透明。

2.4 智能体沙箱的网络隔离

AI 智能体平台通常采用容器 (如 Docker[42])、

轻量虚拟机 (Firecracker[43]、gVisor[44]) 等技术实现沙箱隔离。例如, OpenHands^[24] 基于容器为智能体提供 bash shell、浏览器和代码执行环境, SWE-agent^[21] 在沙箱化环境中提供 Agent-Computer Interface 供智能体访问代码仓库和终端。这些技术主要关注计算和存储资源的隔离, 而对网络出口的差异化路由支持有限。目前业界通常采用以下方式解决智能体的外部 API 访问问题: 1) 为沙箱配置统一的 HTTP 代理, 但这要求智能体代码显式使用代理; 2) 为沙箱建立 VPN 连接, 但每个服务需要独立的隧道; 3) 使用网络命名空间 (network namespace) 和虚拟以太网 (veth) 配合 iptables 规则进行流量管控, 但规则的维护复杂度高且无法直接利用域名信息。星汉系统可以部署在沙箱的网络命名空间出口处, 透明地根据服务域名将流量路由到不同出口, 与现有的沙箱隔离技术天然兼容。

3 星汉系统的设计

3.1 概述

星汉系统的透明按名路由设计主要由三个组件构成: 名字选择、连接代理和名字解析。

星汉系统专注于 TLS 流量。星汉系统利用 TLS 服务器名称指示 (Server Name Indication, SNI) 扩展中的服务名和端口 (以下简称“服务名”) 来标识每个连接。TLS SNI 最初被引入是为了在单一主机上托管多个站点时分发正确的证书, 但它恰好可以作为服务标识符使用。事实上, HTTPS (包含 TLS、TCP 和 IP) 正在成为当今互联网架构新的“细腰” (narrow waist)[45]。截至 2026 年 3 月, HTTPS 占有 Web 流量的 96%[46], 75.3% 的网站支持现代 TLS 1.3 协议[47]。在 AI 智能体的多源服务接入场景中, 主流大模型 API 提供商 (OpenAI[48]、Anthropic[49]、Google Gemini[50] 等) 均强制要求使用 HTTPS 访问; 第三方工具调用 (如 GitHub[51] 等) 也几乎全部通过 HTTPS 对外提供服务, 这使得基于 TLS SNI 的按名路由设计具有广泛的适用性。

为了代理 TLS 流量并提取服务名, 我们提出了四路 TCP-TLS 联合握手代理。由于 TLS SNI 携带在 Client Hello 数据包中 (TCP 握手完成后的第一个数据包), 代理在收到 Client Hello 之前无法决定应当将 TCP 流转发到何处。因此, 代理需要暂时挂起与客户端的 TCP 握手, 从 Client Hello 中提取 SNI 读取服务名, 然后才能向真实服务器发起新的握手。

为高效地根据服务名将连接重定向到指定的真实服务器, 我们引入了延迟内核态名字解析机制。与 DNS 在用户空间于连接建立前完成名字解析不同,

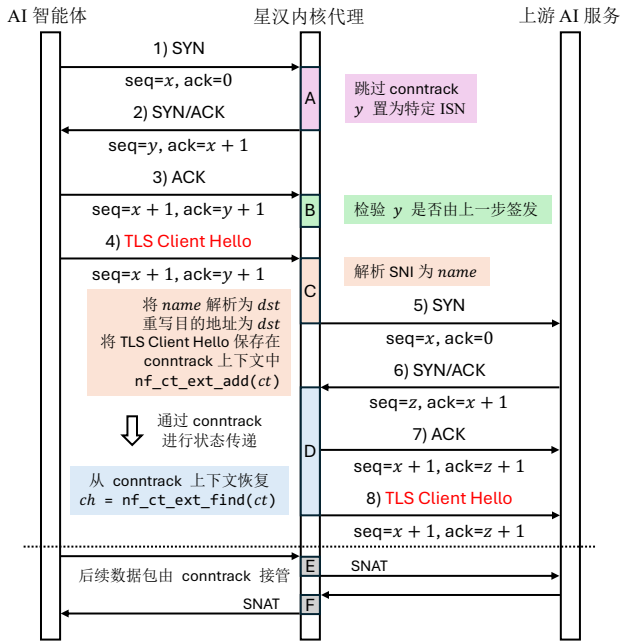


Figure 3 Workflow of the 4-way TCP-TLS joint handshake

proxy

图 3 四路 TCP-TLS 联合握手代理的工作流

星汉系统将此过程推迟到内核的数据包转发路径中执行。通过在内核态数据结构中维护服务名到地址的映射，星汉系统在提取服务名后立即解析目的 IP 地址。这种内核态设计避免了用户空间查找的性能损失，同时确保路由决策在内核态名字查找之后即时做出。

图 2 展示了星汉系统支持的典型 AI 智能体多源 API 接入场景。一个 AI 智能体沙箱发起对 <https://bedrock.us-west-1.amazonaws.com> 的 HTTPS 请求。部署在沙箱的网络出口，运行在内核态的星汉系统网关拦截该请求，建立四路握手代理会话，解析 TLS SNI 获取服务名 `bedrock.us-west-1.amazonaws.com:443`，并执行路由时名字解析，将目的地址改写为位于 `us-west-1` 地域的 AWS 出口网关的地址。当流量到达出口侧星汉系统代理后，应用同样的代理和名字解析逻辑，此时将目的地址改写为真实 AWS Bedrock 服务器的内部地址。当 API 服务器处理请求并生成响应时，两级星汉系统代理（出口侧和网关侧）反向应用 SNAT 规则，将响应数据包透明地返回给智能体。由于真实服务器的内部地址从未暴露给智能体沙箱或网关侧代理，真实服务器的地址变更不会影响数据包的正确路由。对于受星汉系统转发的域名，用户态 DNS 必须始终返回一个结果（无论其是否正确），以确保请求能够到达内核进行处理而不至于在用户态就失败；对于不匹配内核名字规则集的域名，星汉系统不执行任何改写，按原样转发数据包。

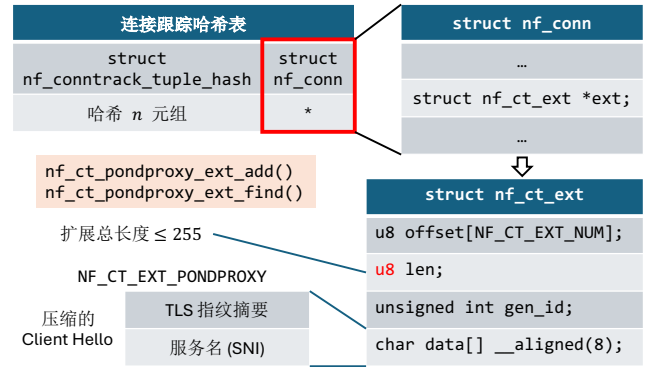


Figure 4 Handshake state passing mechanism based on conntrack extensions

图 4 基于连接跟踪扩展的握手状态传递机制

3.2 四路 TCP-TLS 联合握手代理

我们提出四路 TCP-TLS 联合握手代理来实现内核态的按名路由，如图所示。四路握手代理实现为 Linux 内核 netfilter[52] 的扩展。我们的四路握手可分为 A、B、C、D 四个握手阶段，以及两个握手后的 SNAT 阶段 E 和 F，如图 3 所示。星汉系统所提出的四路握手代理可以应用于代理服务器上 FORWARDING 链的 filter 表，以及客户端上 OUTPUT 链的 filter 表。为简洁起见，本节仅描述代理服务器场景；客户端场景遵循相同的原理。

阶段 A: SYN 劫持。在 TCP 三次握手期间，由于代理尚未读取到服务名，无法确定应当将 SYN 数据包路由到何处。因此，需要临时劫持 (NF_STOLEN) SYN 数据包。与 SYNPROXY[53] 设计类似，我们的 ASTRAPROXY 内核模块完全管理整个握手过程。为此，对所有入站 SYN 数据包禁用连接跟踪 (connection tracking)，以防止内核的连接跟踪子系统抢占星汉代理对握手的控制权。由于缺少连接跟踪条目，TCP 三次握手需要设计为无状态的。因此，代理需要根据 SYN Cookie 算法计算初始序列号 (Initial Sequence Number, ISN)，并向客户端发送 SYN/ACK 响应。

阶段 B: ACK 验证。由于 TCP 握手过程是无状态的，代理无法从连接跟踪条目中检索前两步握手的信息。相反，它需要从 ACK 数据包的 ack 号码中提取阶段 A 设置的 SYN Cookie，并验证该 Cookie 是否由自身签发。该机制同时可用于防御 SYN 洪泛攻击 (SYN Flooding)。此时，代理已完成与客户端的 TCP 三次握手，但仍需等待客户端发送 TLS 握手数据包后，才能继续进行与真实服务器的握手。

阶段 C: 名字提取与目的地址确定。对于 TLS 请求，在 TCP 握手完成后，客户端继续进行 TLS 握手，其第一个数据包即为 Client Hello。收到 Client Hello 后，代理应从 TLS SNI 扩展中提取

表 1 星汉系统基数树的符号与术语定义
Table 1 Notation and terminology for the ASTRA radix tree

符号	定义
nil	空字符。
$*$	通配符。 $*$ 可以匹配 FQDN 的任意非空前缀。它只能出现在名字规则中的第一个标签位置，并且最多只能出现一次。
Σ	FQDN 字母表，包含标签字符和分隔符点号 (.)。标签字符包括字母（不区分大小写）、数字和连字符。 $\Sigma = \{a, \dots, z\} \cup \{0, \dots, 9\} \cup \{-\} \cup \{.\}$
$ \Sigma $	FQDN 字母表 Σ 的大小。 $ \Sigma = 38$
Σ^*	由字母表 Σ 中的字符所构成的、最大长度为 255 的所有 FQDN 的集合。
$\sigma \in \Sigma^*$	一个查询的 FQDN
Π	名字规则字母表。在 Σ 中添加 $*$ 以实现后缀匹配。 $\Pi = \Sigma \cup \{*\}$
$ \Pi $	名字规则字母表 Π 的大小。 $ \Pi = 39$
Π^*	由字母表 Σ 中的字符所构成的所有名称规则的集合
$\pi \in \Pi^*$	一条名字规则
$\pi \sqsupset \sigma$	名字规则 π 是 FQDN σ 的一个后缀
$\pi \sqsubset \sigma$	反转的名字规则 π 是反转的 FQDN σ 的一个前缀
T_r^n	星汉系统存储名字规则的基数树，其中 r 为基数， n 为叶子节点个数

服务名。然后，检查该服务名是否匹配名字规则集中的条目，从而确定当前连接的目的地址。最后，代理向确定的目的地址发送 SYN 数据包，发起与真实服务器的 TCP 握手。

值得注意的是，当代理向真实服务器发送 SYN 数据包时，必须同时建立连接跟踪条目。这是因为四路握手之后的流量仍需由内核连接跟踪处理，因此需要在确定目的地址后立即建立连接跟踪条目。此外，与代理发送给客户端的 SYN/ACK 数据包（图中的步骤 1）不同，代理发送给真实服务器的 SYN 数据包（图中的步骤 5）不能再携带任何状态信息。原因在于步骤 5 中 SYN 数据包的序列号必须与步骤 1 中客户端发送给代理的 SYN 数据包的序列号保持一致，以确保后续客户端与服务器之间的 TCP 序列号同步。由于步骤 5 的 SYN 数据包不能包含额外信息，代理将把 Client Hello 信息有状态地存储在连接跟踪条目中，以备后续恢复使用。

阶段 D: ACK 和 TLS Client Hello 发送。代理收到真实服务器的 SYN/ACK 后，应向真实服务器

回复 ACK 数据包以完成 TCP 握手。由于步骤 5 的 SYN 数据包和步骤 6 的 SYN/ACK 数据包属于同一连接跟踪条目，代理可以提取阶段 C 存储的、重建 Client Hello 数据包所需的信息，并向真实服务器发送代理后的 Client Hello 消息。此步骤之后，TCP-TLS 联合握手完成，后续流量将继续由内核连接跟踪管理。

阶段 E 和 F 执行 SNAT。从连接跟踪的角度来看，四路握手过程本质上执行了一次手动 DNAT 操作。为确保数据包能够正确路由到代理后方的真实服务器（无论其在同一网络还是不同网络），需要分别对出站和入站数据包执行 SNAT/MASQUERADE 操作。SNAT 始终在 POSTROUTING 链的 nat 表中执行，发生在星汉代理在 filter 表中生效之后。对于来自客户端的出站数据包，SNAT 将数据包的源 IP 地址设置为代理在面向真实服务器的接口上的 IP 地址。对于来自真实服务器的入站数据包，SNAT 将数据包的源 IP 地址设置为代理在面向客户端的接口上的 IP 地址。不难观察到，ASTRAPROXY 始终作用在 filter 表而非通常用于 DNAT 操作的 nat 表。这是因为 ASTRAPROXY 需要调用

`ip_route_me_harder()` 和 `ip_local_out()` 来发送新数据包，而 nat 表不提供此功能。

考虑四路握手状态维护。握手过程涉及两种不同类型的状态维护。从阶段 A 到阶段 B 的状态传递通过 SYN Cookie 实现，因此代理不需要在内部维护任何按连接的状态。相比之下，从阶段 C 到阶段 D 的状态传递依赖于连接跟踪记录的扩展，如图所示。在阶段 C，代理必须将所有准备在阶段 D 发送给真实服务器的 ACK、Client Hello 等握手所需的信息持久化下来以便后续恢复。发送 ACK 是直接的，但重建 Client Hello 涉及保存 SNI 扩展和客户端的 TLS 指纹[54][55]。由于不同客户端支持的 TLS 密码套件(cipher suites)和其他握手参数各不相同，代理在阶段 D 向真实服务器发送 Client Hello 时必须精确地重现这些特征，以避免因指纹不匹配导致握手失败。为此，我们扩展了连接跟踪记录，增加了自定义扩展字段来存储请求的 SNI 和对应的 TLS 指纹条目索引。

3.3 延迟内核态名字解析

四路 TCP-TLS 联合握手代理实现了内核级别的按名路由。除了代理连接之外，星汉系统还需要在路由时将名字解析为具体的目的 IP 地址，这被称为延迟内核态名字解析。在此设计中，“延迟”意味着与用户空间发生的域名解析不同，服务名与其目的 IP 之间的绑定关系在数据包路由的时刻才被确定，

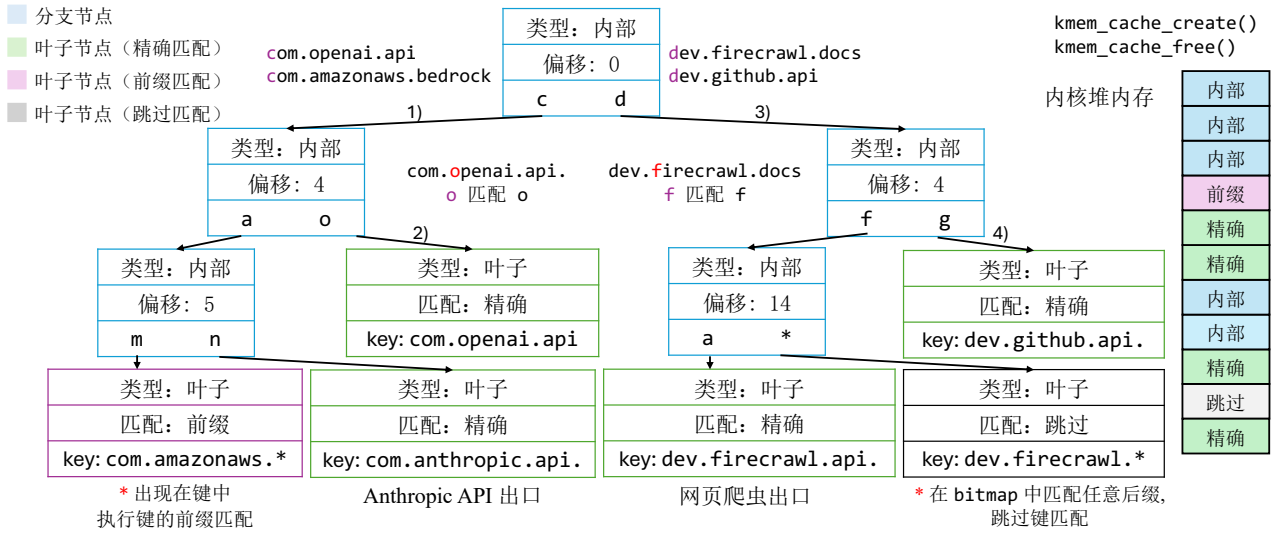


Figure 6 Example of the ASTRA radix tree and its matching process: a radix tree containing 6 name rules, illustrating exact match, prefix match, and skip match lookups, with the corresponding kernel heap memory layout shown on the right

图 6 星汉系统基数树示例及匹配过程: 包含 6 条名字规则的基数树结构, 展示了精确匹配、前缀匹配和跳过匹配三种查找方式, 右侧为对应的内核堆内存布局

而非在用户空间预先解析。“内核态”则表示此查找过程完全在内核的数据包处理路径内执行。在 AI 智能体多源服务接入的特定场景中, 星汉系统的名字解析主要关注以下两类规则:

1) 将特定域名 (如 `api.openai.com`) 映射到特定 IP 地址;

2) 将具有给定后缀的所有域名 (如 `*.amazonaws.com`) 映射到特定 IP 地址。

值得注意的是, 此处的“后缀”不一定是顶级域名 (Top-Level Domain, TLD), 而是字面字符串后缀。得益于域名的结构化语义, 这种后缀匹配使星汉系统能够将属于特定服务提供商的所有连接路由到同一出口 IP 地址。

为同时满足精确匹配和后缀匹配的需求, 星汉系统采用专门设计的基数树来存储和查询域名规则-目的 IP 的映射记录。为确保匹配始终从字符串的开头开始, 我们在匹配过程中使用反转的完全限定域名 (Fully Qualified Domain Name, FQDN) 和反转的名字规则, 即反转域名标签的顺序。为更严格地形式化星汉系统所采用的基数树, 我们在表 1 中总结了符号和术语。

在星汉系统的基数树中, 每个内部节点维护以下三个字段:

1) **offset**: 对于查询的 FQDN, **offset** 用于指定在选择下一个子分支时应检查的字符索引。随着查询在树中的深度增加, **offset** 值始终递增。

2) **bitmap**: 固定长度的 **bitmap**, 精确指示当前节点存在哪些出方向的分支 (即 $\Pi \cup \{nil\}$ 中的哪些

字符)。**bitmap** 的大小为 40, 其中每一位对应 $\Pi \cup \{nil\}$ 中的一个字符。如果存在符号 $c \in \Pi \cup \{nil\}$ 的分支, 则与 c 关联的位被设置为 1。

3) **children**: 指向子节点引用数组的指针。子节点按照 **bitmap** 中 1 位置的顺序排列 (即按位索引递增顺序)。

每个叶子节点维护以下三个字段:

1) **key**: 名字规则 π , 可以是反转的 FQDN 或反转的 FQDN 后缀模式。 $key \in \Pi$ 。

2) **value**: 与名字规则关联的目的 IP 地址。

3) **match**: 匹配类型, 分为以下三类:

a. 精确匹配: 叶子的 **key** 是一个反转的 FQDN。检查 **key** 是否与查询的 FQDN 完全相等;

b. 前缀匹配: 叶子的 **key** 是反转 FQDN 的前缀 (即 FQDN 的后缀)。检查 **key** 是否是查询 FQDN 的后缀;

c. 跳过匹配: 叶子是通过其父节点 **bitmap** 中的通配符 (*) 分支到达的; 由于父节点通配符匹配任意字符序列, 跳过匹配直接返回 **value**。

图 6 展示了一个包含 6 条规则的星汉系统基数树示例及其匹配过程。可以直观地证明星汉系统基数树满足以下性质: 对任意 $\sigma \in \Sigma^*$:

1) 若对所有 $\pi \in T$, π 均无法匹配 σ , 则查询算法返回 **NULL**;

2) 若存在 $\pi \in T$ 可以匹配 σ , 则查询算法返回具有最高优先级的名字规则 π^* 的目的 IP。优先级顺序为: 精确匹配优先于所有其他匹配, 在所有非精确匹配中, 前缀更长者优先。

我们分析星汉系统基数树查询的时间和空间复杂度。假设每个查询的 FQDN 为 $\sigma \in \Sigma^*$ ，其中 $|\sigma| = m$ 。设基数树 T_r^n 包含 n 条名字规则（即 n 个叶子节点）。查询从根节点向下经过零个或多个内部节点，最终到达恰好一个叶子节点。在每个内部节点处，读取字符 $\sigma[\text{offset}]$ 的时间为 $O(1)$ 。使用节点的 `bitmap` 选择下一个子指针也是 $O(1)$ 的时间复杂度。对于精确匹配或前缀匹配的叶子，验证 σ 是否匹配存储的 `key` 需要 $O(m)$ 的时间。对于跳过匹配的叶子节点，匹配测试为 $O(1)$ 。如果最终叶子节点的深度为 d ，则每次查询的总时间为 $O(d + m)$ 。在具有 n 个叶子的 r 叉基数树 T_r^n 中，深度 d 满足：

$$\Omega(\log_r n) \leq d \leq O(n)$$

因此，每次查询的总时间复杂度具有以下界：最佳（平衡）情况： $\Omega(\log_r n + m)$ ，最坏（退化）情况： $O(n + m)$ 。值得注意的是，星汉系统基数树的基数 $r = 40$ ，这是一个相对较大的基数，因此对于典型的规则集大小 n ， $\log_r(n)$ 的值变化很小。这意味着星汉系统基数树随规则数量 n 的增长具有良好的可扩展性。我们将在第 4.4 节中详细考察这一问题。

星汉系统基数树的空间复杂度由所有叶子和所有内部节点占用的总空间给出。设 b 为内部节点数量，每个内部节点 I_i 的出度为 $\deg(I_i)$ 。树的总出度为 $\sum_i \deg(I_i)$ ，等于总边数 $b + n - 1$ 。 $\sum_i \deg(I_i)$ 满足 $2b \leq \sum_i \deg(I_i) \leq \sigma b$ 。因此：

$$\frac{n-1}{\sigma-1} \leq b \leq n-1$$

所以星汉系统基数树的空间复杂度为 $\Theta(n)$ 。我们将在第 4.4 节中验证此结果。

在星汉系统中，基数树的节点在内核堆中分配。为使用户态程序管理这些节点，星汉系统提供了名为 `ASTRAPROXY` 的 `xtables` 扩展，通过调用 `kmem_cache_alloc()` 和 `kmem_cache_free()` 调用内核的 `slab` 分配器进行内存分配和释放。用户态应用程序只需调用 `ASTRAPROXY` 扩展即可添加或删除基于名字的规则。从用户态程序的角度来看，管理星汉系统的名字规则与管理 `iptables` 中的 `DNAT` 规则一样简单。唯一的区别是匹配参数从目的 IP 地址变更为名字规则。

3.4 实现

我们在配备 Intel Xeon Gold 6442Y 48 核 CPU 和 256 GB 内存的 Linux 服务器上实现了星汉系统，操作系统为 Arch Linux（内核版本 v6.19.6-arch1），使用 `iptables v1.8.11 (nf_tables)`。整个系统使用 C 语言编写。内核空间中，约 1500 行 C 代码处理四路 TCP-TLS 联合握手，约 1200 行 C 代码实现

基数树名字匹配；用户空间中，约 500 行 C 代码实现新的 `xtables` 目标扩展。

4 实验与评估

星汉系统旨在为 AI 智能体沙箱提供透明、高性能的多源服务接入能力。为验证这些特性，我们首先在模拟的多源 API 接入环境中部署星汉系统，验证其能够根据服务名正确地将流量路由到不同的网络出口。在此基础上，我们通过一系列实验评估星汉系统的性能，并回答以下问题：

1) 星汉系统的内核态四路 TCP-TLS 联合握手代理在数据转发吞吐量和每秒处理请求数（Queries per Second, QPS）方面与现有的用户态代理（HAProxy、Envoy）相比表现如何？

2) 在不同消息大小下，星汉系统将 TLS 服务访问的端到端延迟降低了多少？

3) 星汉系统的延迟内核态名字解析机制在名字规则数量增长时的内存使用和查找开销如何变化？

4) 在模拟 AI 智能体多 API 并发调用模式下，星汉系统相比用户态代理方案的优势如何？

4.1 实验环境

我们的实验测试床由一台物理服务器和多台 KVM 虚拟机组成。物理服务器配备 48 核 CPU 和 256 GiB 内存，部署星汉系统或对比方案的用户态代理。我们使用一台 16 核、32 GiB 内存的 KVM 虚拟机作为客户端生成 HTTPS 流量，另外 20 台相同配置的 KVM 虚拟机模拟真实 API 服务器，每个服务器执行 TLS 终止并处理 HTTPS 请求。客户端-代理和服务器-代理之间的链路由 Mellanox ConnectX-6 网卡提供，工作在以太网模式下，速率为 200 Gbps。

作为负责 TLS 终止和请求处理的后端服务，我们使用 `nginx`[19] 提供不同大小的静态文件，以模拟真实的 API 响应流量。为更贴近 AI 智能体的 API 调用场景，我们重点关注 1 KiB 至 64 KiB 范围的小消息（典型的 API JSONRPC 请求/响应大小），同时也测试了更大的消息（至 8 MiB）以评估大数据传输场景（如文件下载 API）。我们使用 Google Trust Services 签发的 ECDSA (secp256r1 曲线) 服务器证书。

对比基线方面，我们选择以下两个广泛使用的用户态代理：

1) HAProxy^[18]：业界广泛使用的 L4/L7 负载均衡器之一，以高性能著称。我们使用 HAProxy 3.3.5 版本，配置为 TCP 模式，利用 `req.ssl_sni` ACL 提取 TLS SNI 进行按名路由。

2) Envoy^[20]：Istio[32] 服务网格的默认数据面代

理,支持丰富的 L7 流量管理功能。我们使用 Envoy 1.33 版本,配置为 TCP 代理模式(TCP proxy filter),利用 filter_chain_match 的 server_names 字段进行 SNI 匹配路由,同时使用 4 个工作线程,per_connection_buffer_limit_bytes 设为 16384 (16 KB)。

需要注意的是,虽然我们使用 HAProxy 和 Envoy 作为对比基线,但它们本身并不以“按名选择网络出口”为设计目标;在此实验中,它们仅执行基于名字的用户态 DNAT 操作。

所有实验使用 wrk 4.2.0[56] 作为负载生成器,每组配置运行 3 次,取中位数以减少测量噪声。

4.2 功能验证

在进行性能测试之前,我们首先验证星汉系统在 AI 智能体多源 API 接入场景中的功能正确性。

我们构建了一个模拟环境,其中包含 4 种不同的网络出口,分别模拟:1) AWS 云内网出口(通往模拟的大模型推理 API);2) 住宅 IP 代理出口(通往模拟的网页抓取 API);3) 专线出口(通往模拟的金融数据 API);4) 企业 VPN 出口(通往模拟的企业内部服务 MCP Server)。每个出口背后部署独立的 API 服务器。星汉系统网关部署在客户端(AI 智能体沙箱)的网络出口处,配置以上名字规则。

我们从客户端依次发起对上述 4 类域名的 HTTPS 请求,并验证:1) 每个请求被正确路由到对应的出口和后端服务器;2) TLS 握手正确完成,客户端收到正确的服务器证书;3) 后续数据传输正常,无数据损坏或丢失;4) 多个域名的请求可以并发执行,互不干扰。星汉系统在所有测试中均正确完成了按名路由功能。

4.3 端到端性能

4.3.1 端到端吞吐量

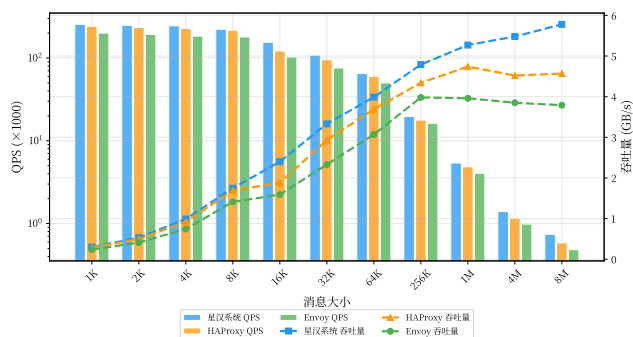


Figure 7 Comparison of QPS and throughput among ASTRA, HAProxy, and Envoy under varying message sizes

图 7 不同消息大小下星汉系统、HAProxy 和 Envoy 的 QPS 与吞吐量对比

我们使用两个指标评估数据包处理能力:每秒请求数(QPS)和吞吐量。QPS 由 wrk 报告,吞吐量按照消息大小乘以 QPS 计算。我们首先评估不同消息大小下各方案的数据包处理能力。客户端使用 wrk 以 8 个线程、256 个并发连接、60 秒持续时间向各代理发送 HTTPS 请求,请求不同大小的静态文件(1 KiB 至 8 MiB)。为聚焦于代理转发性能,名字规则限制为 10 条。图 7 对比了星汉系统、HAProxy 和 Envoy 在不同消息大小下的 QPS 和吞吐量。在 1 KiB 到 8 MiB 的全部负载范围内,星汉系统在 QPS 方面均优于 HAProxy 和 Envoy。在小消息场景下(1 KiB 至 8 KiB),三种方案的 QPS 差距相对较小:星汉系统达到 255K QPS(1 KiB),HAProxy 为 242K (-5.2%), Envoy 为 200K (-21.6%)。随着消息大小的增长,差距逐渐扩大。在 16 KiB 和 32 KiB 这两个对 AI 智能体 API 调用最为典型的消息大小下,星汉系统的 QPS 分别超过 HAProxy 27.16% 和 13.46%,超过 Envoy 50.74% 和 42.91%。

上述性能差距的根本原因在于数据路径的差异。星汉系统在内核的 netfilter 框架中完成地址改写,数据包始终在内核空间的套接字缓冲区中流转,不涉及用户态拷贝或上下文切换。相比之下,HAProxy 和 Envoy 作为用户态进程,每个代理连接需要经历至少两次内核-用户空间的数据传输:一次从客户端套接字读取到用户态缓冲区,一次从用户态缓冲区写入到后端套接字。在我们的实验中,HAProxy 平均每秒产生约 10.9K 次非自愿上下文切换,Envoy 约 12.3K 次,而星汉系统的数据路径中不存在额外的上下文切换。

4.3.2 端到端延迟

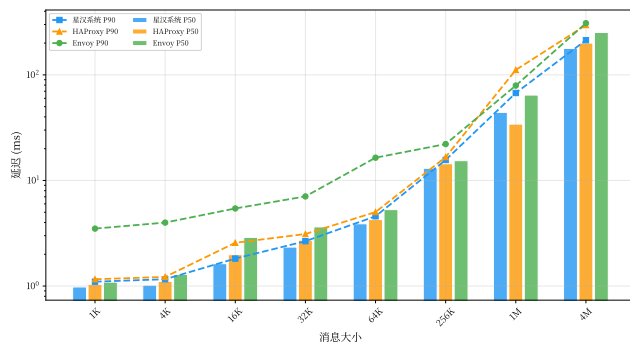


Figure 8 Comparison of P50 and P90 end-to-end latency among ASTRA, HAProxy, and Envoy under varying message sizes

图 8 不同消息大小下星汉系统、HAProxy 和 Envoy 的 50 百分位数延迟与 90 百分位数延迟对比

进行离线分析。图 8 对比了星汉系统、HAProxy 和 Envoy 在不同消息大小下的第 50 和第 90 百分位端到端延迟。在小消息场景下（1 KiB 至 8 KiB），星汉系统的 P50 延迟维持在 0.98~1.13 ms，HAProxy 为 1.04~1.16 ms，两者差距在 6%~8% 之间。Envoy 的 P50 延迟为 1.09~1.30 ms，略高于 HAProxy；在 16 KiB 和 32 KiB 消息大小下，延迟差距进一步扩大。星汉系统的 P50 延迟分别为 1.63 ms 和 2.34 ms，HAProxy 为 1.98 ms 和 2.72 ms（分别高出 21.5% 和 16.2%），Envoy 为 2.70 ms 和 3.63 ms（分别高出 65.6% 和 55.1%）。除了前文讨论的上下文切换和内存拷贝开销外，星汉系统的这一改善还源于用户态代理较长的代码处理路径（如 HAProxy 的线性时间 ACL 评估和 Envoy 的 filter chain 匹配）。

4.4 名字规则可扩展性

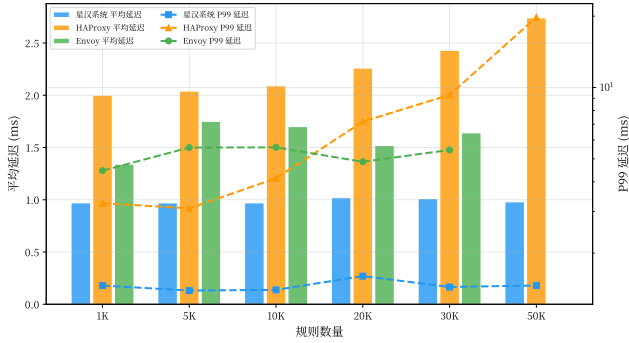


Figure 9 Memory usage and average latency of ASTRA, HAProxy, and Envoy as the number of name rules scales from 1K to 50K

图 9 名字规则数量从 1K 增长到 50K 时星汉系统、HAProxy 和 Envoy 的内存占用与平均延迟变化

除了端到端的吞吐量和延迟，延迟名字解析的可扩展性也是衡量星汉系统性能的重要指标。我们使用 Cisco Umbrella Popularity List[58] 作为域名语料库，以模拟真实生产流量多源服务的多样性和规模。我们从该列表中选取一些子集作为精确匹配规则和后缀匹配规则，生成 wrk 客户端请求，其 Host 头部和 URL 依次使用子集中的域名，模拟 AI 智能体访问多种不同服务的场景。我们评估名字规则数量从 1,000 增长到 50,000 时对转发性能和延迟的影响。实验使用固定的 2 KiB 消息大小（模拟典型 API JSONRPC 响应），wrk 参数与前述实验一致（8 线程、256 并发连接、60 秒持续时间）。

在星汉系统中，名字规则存储在内核态基数树中。在 HAProxy 中，规则编码为配置文件中的 use_backend ACL 条目，使用 req.ssl_sni -i 进行大小写不敏感的精确匹配。在 Envoy 中，规则通过 filter_chain_match 的 server_names 配置。我们测量两个指标：1) 名字规则数据结构的内存占

用；2) 平均每请求端到端延迟。对于 POND 基数树，通过读取 /proc/slabinfo 中对应的 slab 条目获取内存使用量。对于 HAProxy 和 Envoy，采用差分测量法：用加载规则集后的驻留内存减去空规则时的驻留内存。

图 9 绘制了两个指标随规则集大小的变化。星汉系统的内存增长远慢于 HAProxy 和 Envoy。在请求延迟方面，HAProxy 和 Envoy 均随 ACL/规则集增长呈近似线性增加，而星汉系统的平均端到端延迟基本保持平坦。星汉系统的 QPS 在 1K 至 50K 规则范围内基本保持恒定（262K~251K，波动仅 4.6%），平均延迟稳定在 0.97~1.02 ms。而 HAProxy 和 Envoy 的表现则呈现明显的线性退化趋势。特别地，Envoy 在 50K 条 filter chain 规则下启动失败，反映

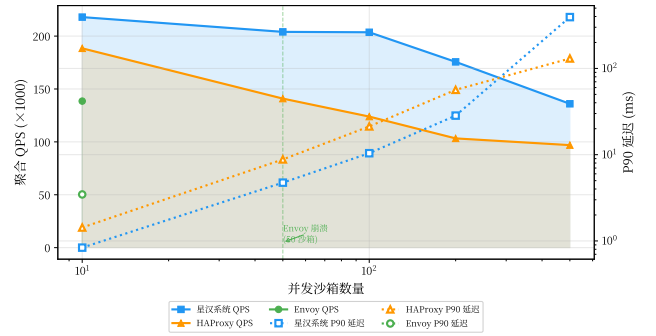


Figure 10 Aggregate QPS and P90 latency under simulated AI agent multi-API concurrent workloads with varying numbers of concurrent sandboxes

图 10 模拟 AI 智能体多 API 并发调用场景下的聚合 QPS 与 90 百分位数延迟随并发沙箱数量的变化

了其配置解析器在大规则集下的局限性。这是因为星汉系统基数树的查找复杂度为 $\Omega(\log_r n + m)$ ，其中 m 为查询域名长度。可以看到，由于 $r = 40$ 使得树深度仅为 4-5 层。几乎恒定的延迟和较低的内存增长共同表明星汉系统的延迟名字解析设计具有良好的可扩展性。

4.5 模拟 AI 智能体多 API 并发调用

为更真实地评估星汉系统在 AI 智能体场景中的表现，我们设计了一个多沙箱多域名并发调用实验。

我们模拟了一个多租户 AI 智能体平台的典型工作负载。每个 AI 智能体沙箱由 4 个并发 wrk 进程组成，分别访问 4 个不同的 API 域名，模拟 AI 智能体同时调用的 4 类服务，每个域名对应不同的后端服务器。请求的负载大小在 1 KiB 至 16 KiB 之间随机分布（模拟 API 的 JSONRPC 请求/响应），并发沙箱数量从 10 逐步增加到 1000。在星汉系统中，我们配置按名路由规则将 4 个域名路由到不同的后端组；在 HAProxy 和 Envoy 中，我们分别使用

对应的 SNI 匹配规则实现等效路由。

图 10 对比了在不同并发沙箱数量(10 至 1000)下三种方案的性能。在低并发场景下(10 个沙箱, 40 个进程), 星汉系统达到 218K QPS, HAProxy 为 188K (-13.7%), Envoy 为 139K (-36.2%)。随着沙箱数量的增加, 星汉系统展现出显著的优势: 在低并发场景下(10 个沙箱, 40 个进程), 星汉系统达到 218K QPS, 较 HAProxy 高 16.0%, 较 Envoy 高 56.8%, P90 延迟为 0.84ms, 分别低于 HAProxy 的 1.43 ms 和 Envoy 的 3.21ms。随着并发规模增长, 星汉系统的优势进一步扩大: 100 个沙箱时 QPS 较 HAProxy 高 64.5%, P90 延迟仅为 HAProxy 的 48.8% (星汉系统为 10.4 ms, HAProxy 为 21.3 ms), Envoy 在此规模下已崩溃退出; 200 个沙箱时 QPS 较 HAProxy 高 70.9%, P90 延迟优势保持在 49.7% (星汉系统为 28.4 ms, HAProxy 为 56.5 ms)。这是因为在高并发场景下, 用户态代理的上下文切换和连接管理开销被急剧放大, 而星汉系统的内核态处理路径在并发度增加时展现出更好的可扩展性。

5 讨论

5.1 名字偏好

尽管我们在实验中仅使用 HTTPS 流量评估星汉系统的性能, 但星汉系统架构和四路 TCP-TLS 联合握手代理适用于所有基于 TLS 的流量, 包括 FTPS、PostgreSQL over SSL 等。事实上, 同一方法可以适配任何在握手过程中携带名字信息的基于 TCP 的协议。一个典型例子是 SSH: 客户端在连接开始时发送的协议版本交换字符串(如 SSH-2.0-OpenSSH_10.0)可以被用来携带名字信息。具体来说, 可以通过配置 SSH 客户端(如使用 `ssh_config` 中的 `VersionAddendum` 选项)追加形如 `@target.svc` 的后缀。星汉系统可以拦截此版本字符串, 提取目标名字, 并在加密会话开始前将连接路由到对应的后端。在 AI 智能体场景中, 这意味着星汉系统不仅能处理 HTTPS API 调用, 还能处理智能体通过 SSH 访问远程服务器等场景。此外, 即使启用了加密客户端 Hello (Encrypted Client Hello, ECH) [59], [60], 星汉系统仍可基于未加密的外层 SNI (Outer SNI) 进行路由。

5.2 安全性考量

四路 TCP-TLS 联合握手的引入不会损害 TLS 的安全模型。星汉系统仅作为 TCP 层的透明转发代理, 不终止 TLS 连接。星汉系统只检查初始的

Client Hello 数据包以提取 SNI, 随后在客户端和真实服务器之间原样转发所有 TLS 数据包。由于密码学参数是端到端协商的, 星汉系统无法作为中间人解密或篡改加密的应用数据, 从而保持了 TLS 会话的完整性。此外, 在内核中部署名字解析逻辑需要严格的安全保障。星汉系统基数树的遍历深度受域名最大长度限制, 确保了确定性的查找时间, 防止算法的拒绝服务攻击。内存通过内核 slab 分配器进行管理并具有适当的隔离, 对配置接口的访问仅限于拥有 `CAP_NET_ADMIN` 权限的用户。

5.3 eBPF 加速

一些研究提出利用 eBPF 重新实现部分网络功能以提升性能[61], [62]。然而需要注意的是, 在 TLS Client Hello 数据包到达之前, 代理需要挂起前置的 TCP 握手(因为没有名字就无法确定真实服务器的 IP 地址)。对于这种有状态的连接操作, eBPF/XDP 本身是不足够的: 还需要内核空间或用户空间组件的支持来执行数据包劫持、连接状态维护和数据包发送。不过, 其他研究[41], [63], [64], [65]已经表明, 代理过程中的无状态部分可以通过在 XDP 中发送 SYN Cookie 来进行卸载。将这一思想应用于星汉系统可能带来进一步的性能提升。

5.4 AI 智能体沙箱的多租户隔离

在多租户 AI 智能体平台中, 不同用户的沙箱可能需要不同的名字路由规则(例如, 不同用户有不同的 API 密钥和接入方式)。星汉系统当前使用全局的内核态名字规则表, 若需要按沙箱隔离规则, 可以结合 Linux 网络命名空间(network namespace)实现: 为每个沙箱创建独立的网络命名空间, 并在每个命名空间中部署独立的星汉系统实例和规则集。由于星汉系统的内核态设计, 其资源占用远低于用户态代理, 使得这种按沙箱部署在实践中是可行的。

6 结论

本文针对 AI 智能体多源服务接入场景中的差异化路由需求, 提出了星汉按名路由系统。AI 智能体在执行任务过程中需要同时访问分布在不同网络域的多个 API 服务, 而这些服务各自有着不同的网络可达性约束。传统的隧道方案、策略路由方案 and 用户态代理方案在可扩展性、动态适应性和性能方面各有不足。星汉系统通过利用 TLS SNI 扩展字段中携带的服务域名作为路由依据, 将路由决策从 IP 地址层提升到名字层, 从根本上解决了 IP 地址无法稳定标识服务的问题。

星汉系统包含三项关键技术。首先，跨层按名路由方案以 TLS 服务名和 TCP 端口的组合作为服务标识符，实现了服务标识与网络位置的解耦。其次，四路 TCP-TLS 联合握手代理在内核态透明地拦截 TCP 和 TLS 握手并提取服务名，无需修改客户端应用或网络基础设施。最后，延迟内核态名字解析基于定制的星汉系统基数树实现了高效的名字到地址转换，支持精确匹配和后缀匹配两种规则类型。整个处理过程在内核空间完成，消除了上下文切换和用户态内存拷贝的开销。

实验结果表明，在不同消息大小下，星汉系统的端到端 QPS 在 16 KiB 和 32 KiB 典型 API 负载下分别超过 HAProxy 27.16% 和 13.46%，超过 Envoy 50.74% 和 42.91%；P50 延迟分别低于 HAProxy 17.68% 和 13.97%，低于 Envoy 43.60% 和 35.54%；峰值吞吐量达到 5.78 GB/s，较 HAProxy 高 20.9%，较 Envoy 高 34.4%。在名字规则可扩展性方面，星汉系统在规则数从 1K 增长到 50K 时平均延迟波动仅 4.6%，而 HAProxy 的 P99 延迟在 30K 规则时恶化 186.5%，Envoy 在 50K 规则时启动失败。在模拟 AI 智能体多 API 并发调用场景中，星汉系统在 100 个并发沙箱下 QPS 较 HAProxy 高 64.5%，P90 延迟仅为 48.8%，而 Envoy 在 50 个沙箱时即崩溃退出。上述结果验证了星汉系统的内核态按名路由方案在吞吐量、延迟、可扩展性和稳定性四个维度上相对于用户态代理方案的全面优势。

作者贡献声明：

参考文献

- [1] L. Wang *et al.*, “A survey on large language model based autonomous agents,” *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.
- [2] S. Yao *et al.*, “React: Synergizing reasoning and acting in language models,” in *The eleventh international conference on learning representations*, 2022.
- [3] OpenClaw, “OpenClaw: Your Own Personal AI Assistant.” 2025. [Online]. Available: <https://github.com/openclaw>
- [4] Anthropic, “Claude Code: An Agentic Coding Tool.” 2025. [Online]. Available: <https://github.com/anthropics/claude-code>
- [5] T. Schick *et al.*, “Toolformer: Language models can teach themselves to use tools,” *Advances in neural information processing systems*, vol. 36, pp. 68539–68551, 2023.
- [6] Y. Qin *et al.*, “Toolllm: Facilitating large language models to master 16000+ real-world apis,” *arXiv preprint arXiv:2307.16789*, 2023.
- [7] Mendable, “Firecrawl: Turn Websites into LLM-Ready Data.” 2024. [Online]. Available: <https://github.com/mendableai/firecrawl>
- [8] GitHub, “GitHub.” 2026. [Online]. Available: <https://github.com>
- [9] Anthropic, “Model Context Protocol (MCP).” 2024. [Online]. Available: <https://modelcontextprotocol.io>
- [10] Amazon Web Services, “Amazon Bedrock.” 2026. [Online]. Available: <https://aws.amazon.com/bedrock/>
- [11] Microsoft, “Azure OpenAI Service.” 2026. [Online]. Available: <https://azure.microsoft.com/en-us/products/ai-services/openai-service>
- [12] Google Cloud, “Vertex AI.” 2026. [Online]. Available: <https://cloud.google.com/vertex-ai>
- [13] Manus, “Manus: A General AI Agent.” 2025. [Online]. Available: <https://manus.im>
- [14] J. A. Donenfeld, “WireGuard: Next Generation Kernel Network Tunnel,” in *NDSS*, 2017, pp. 1–12.
- [15] K. Seo and S. Kent, “Security Architecture for the Internet Protocol,” no. 4301. in Request for Comments. RFC Editor, Dec. 2005. doi: 10.17487/RFC4301.
- [16] Amazon Web Services, “AWS IP Address Ranges.” 2026. [Online]. Available: <https://docs.aws.amazon.com/vpc/latest/userguide/aws-ip-ranges.html>
- [17] M. Calder, X. Fan, Z. Hu, E. Katz-Bassett, J. Heidemann, and R. Govindan, “Mapping the expansion of Google’s serving infrastructure,” in *Proceedings of the 2013 conference on Internet measurement conference*, 2013, pp. 313–326.
- [18] HAProxy, “The Reliable, High Performance TCP/HTTP Load Balancer.” [Online]. Available: <https://www.haproxy.org/>
- [19] nginx, “nginx.” [Online]. Available: <https://nginx.org/>
- [20] M. Klein and B. Hochmuth, “Lyft’s Envoy: Experiences Operating a Large Service Mesh,” in *SREcon17 Americas*, USENIX Association, 2017. [Online]. Available: <https://www.usenix.org/conference/srecon17americas/program/presentation/klein>
- [21] J. Yang *et al.*, “Swe-agent: Agent-computer interfaces enable automated software engineering,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 50528–50652, 2024.
- [22] S. Hong *et al.*, “MetaGPT: Meta programming for a multi-agent collaborative framework,” in *The twelfth international conference on learning representations*, 2023.
- [23] H. Chase, “LangChain.” 2023. [Online]. Available:

- <https://github.com/langchain-ai/langchain>
- [24] X. Wang *et al.*, “Openhands: An open platform for ai software developers as generalist agents,” *arXiv preprint arXiv:2407.16741*, 2024.
- [25] Glama, “MCP Servers Registry.” 2026.
- [26] Amazon Web Services, “AWS PrivateLink.” 2026. [Online]. Available: <https://aws.amazon.com/privatelink/>
- [27] M. H. Jhaveri, O. Cetin, C. Gañán, T. Moore, and M. V. Eeten, “Abuse reporting and the fight against cybercrime,” *ACM Computing Surveys (CSUR)*, vol. 49, no. 4, pp. 1–27, 2017.
- [28] A. Viswanathan, E. C. Rosen, and R. Callon, “Multiprotocol Label Switching Architecture,” no. 3031. in Request for Comments. RFC Editor, Jan. 2001. doi: 10.17487/RFC3031.
- [29] C. J. Bernardos, A. Rahman, J.-C. Zúñiga, L. M. Contreras, P. A. Aranda, and P. Lynch, “Network Virtualization Research Challenges,” no. 8568. in Request for Comments. RFC Editor, Apr. 2019. doi: 10.17487/RFC8568.
- [30] Cloudflare, “Cloudflare Access: Zero Trust Network Access.” 2026. [Online]. Available: <https://www.cloudflare.com/products/zero-trust/access/>
- [31] J. Yonan, “OpenVPN: An Open Source VPN Daemon.” 2025. [Online]. Available: <https://openvpn.net/>
- [32] Istio Contributors, “Istio: An Open Platform to Connect, Manage, and Secure Microservices.” 2024. [Online]. Available: <https://github.com/istio/istio>
- [33] P. Francis and R. Gummadi, “IPNL: A NAT-extended internet architecture,” in *Proceedings of the 2001 conference on Applications, technologies, architectures, and protocols for computer communications*, 2001, pp. 69–80.
- [34] R. Moskowitz and M. Komu, “Host Identity Protocol Architecture,” no. 9063. in Request for Comments. RFC Editor, Jul. 2021. doi: 10.17487/RFC9063.
- [35] R. Moskowitz and P. Nikander, “Host Identity Protocol (HIP) Architecture,” no. 4423. in Request for Comments. RFC Editor, May 2006. doi: 10.17487/RFC4423.
- [36] J. J. Garcia-Luna-Aceves, “Name-based content routing in information centric networks using distance information,” in *Proceedings of the 1st ACM Conference on Information-Centric Networking*, 2014, pp. 7–16.
- [37] A. Afanasyev, J. Burke, T. Refaei, L. Wang, B. Zhang, and L. Zhang, “A brief introduction to named data networking,” in *MILCOM 2018-2018 IEEE Military Communications Conference (MILCOM)*, IEEE, 2018, pp. 1–6.
- [38] M. Mahalingam *et al.*, “Virtual eXtensible Local Area Network (VXLAN): A Framework for Overlaying Virtualized Layer 2 Networks over Layer 3 Networks,” no. 7348. in Request for Comments. RFC Editor, Aug. 2014. doi: 10.17487/RFC7348.
- [39] Envoy Project, “Envoy Proxy SNI-Based Routing.” 2025. [Online]. Available: <https://www.envoyproxy.io/docs/envoy/latest/faq/configuration/sni>
- [40] X. Shi, L. He, J. Zhou, Y. Yang, and Y. Liu, “Miresga: Accelerating Layer-7 Load Balancing with Programmable Switches,” in *Proceedings of the ACM on Web Conference 2025*, 2025, pp. 2424–2434.
- [41] S. Yoo, X. Chen, and J. Rexford, “{SmartCookie}: Blocking {Large-Scale} {SYN} Floods with a {Split-Proxy} Defense on Programmable Data Planes,” in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 217–234.
- [42] Docker, Inc., “Docker: Accelerated Container Application Development.” 2025. [Online]. Available: <https://www.docker.com/>
- [43] A. Agache *et al.*, “Firecracker: Lightweight virtualization for serverless applications,” in *17th USENIX symposium on networked systems design and implementation (NSDI 20)*, 2020, pp. 419–434.
- [44] E. G. Young, P. Zhu, T. Caraza-Harter, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, “The True Cost of Containing: A gVisor Case Study,” in *11th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 19)*, Renton, WA: USENIX Association, Jul. 2019. [Online]. Available: <https://www.usenix.org/conference/hotcloud19/presentation/young>
- [45] L. L. Peterson and B. S. Davie, *Computer networks: a systems approach*. Elsevier, 2024.
- [46] Google, “HTTPS encryption on the web.” [Online]. Available: <https://transparencyreport.google.com/https/overview>
- [47] S. S. L. Labs, “SSL Pulse.” [Online]. Available: <https://www.ssllabs.com/ssl-pulse/>
- [48] OpenAI, “OpenAI.” 2026. [Online]. Available: <https://openai.com>
- [49] Anthropic, “Anthropic.” 2026. [Online]. Available: <https://www.anthropic.com>
- [50] Google, “Google Gemini.” 2026. [Online]. Available: <https://gemini.google.com>
- [51] GitHub, “GitHub.” 2026. [Online]. Available: <https://github.com>
- [52] J. Engelhardt and N. Bouliane, “Writing netfilter modules,” *Revised, February*, vol. 7, 2011.

- [53] P. McHardy, “netfilter: implement netfilter SYN proxy.” [Online]. Available: <https://lwn.net/Articles/563151/>
- [54] M. Husák, M. Čermák, T. Jirsík, and P. Čeleda, “HTTPS traffic analysis and client identification using passive SSL/TLS fingerprinting,” *EURASIP Journal on Information Security*, vol. 2016, no. 1, p. 6, 2016.
- [55] J. Althouse, “TLS Fingerprinting with JA3 and JA3S.” [Online]. Available: <https://engineering.salesforce.com/tls-fingerprinting-with-ja3-and-ja3s-247362855967/>
- [56] W. Glozer, “wrk—A HTTP Benchmarking Tool.” 2025. [Online]. Available: <https://github.com/wg/wrk>
- [57] “WireShark.” 2025. [Online]. Available: <https://www.wireshark.org/>
- [58] Cisco, “Cisco Umbrella Popularity List.” 2025. [Online]. Available: <https://s3-us-west-1.amazonaws.com/umbrella-static/index.html>
- [59] E. Rescorla, K. Oku, N. Sullivan, and C. A. Wood, “TLS Encrypted Client Hello,” no. 9849. in Request for Comments. RFC Editor, Mar. 2026. doi: 10.17487/RFC9849.
- [60] C. Huitema, “Issues and Requirements for Server Name Identification (SNI) Encryption in TLS,” no. 8744. in Request for Comments. RFC Editor, Jul. 2020. doi: 10.17487/RFC8744.
- [61] B. Yang *et al.*, “HyperCom: Enabling High Performance and Composable Data Structures for Software Network Functions with eBPF,” in *IEEE INFOCOM 2025-IEEE Conference on Computer Communications*, IEEE, 2025, pp. 1–10.
- [62] M. Butrovich *et al.*, “Tigger: A database proxy that bounces with user-bypass,” *Proceedings of the VLDB Endowment*, vol. 16, no. 11, pp. 3335–3348, 2023.
- [63] M. Mikityanskiy, “Accelerating synproxy with XDP.” [Online]. Available: <https://netdevconf.info/0x15/session.html?Accelerating-synproxy-with-XDP>
- [64] S. F. P. Penkov E. Dumazet, “Issuing SYN cookies in XDP.” [Online]. Available: <https://netdevconf.info/0x14/session.html?talk-issuing-SYN-cookies-in-XDP>
- [65] K. Iwashima, “SYN Proxy at Scale with BPF.” [Online]. Available: <https://lpc.events/event/17/contributions/1645/>